



Article

SG-RAG MOT: SubGraph Retrieval Augmented Generation with Merging and Ordering Triplets for Knowledge Graph Multi-Hop Question Answering

Ahmmad O. M. Saleh ^{1,*} , Gokhan Tur ² and Yucel Saygin ¹ ¹ Computer Science and Engineering, Faculty of Engineering and Natural Sciences, Sabanci University, Istanbul 34956, Turkey; ysaygin@sabanciuniv.edu² Siebel School of Computing and Data Science, The Grainger College of Engineering, University of Illinois Urbana-Champaign, Champaign, IL 61820, USA; gokhan@illinois.edu

* Correspondence: ahmmad@sabanciuniv.edu; Tel.: +90-551-201-9914

Abstract

Large language models (LLMs) often tend to hallucinate, especially in domain-specific tasks and tasks that require reasoning. Previously, we introduced SubGraph Retrieval Augmented Generation (SG-RAG) as a novel Graph RAG method for multi-hop question answering. SG-RAG leverages Cypher queries to search a given knowledge graph and retrieve the subgraph necessary to answer the question. The results from our previous work showed the higher performance of our method compared to the traditional Retrieval Augmented Generation (RAG). In this work, we further enhanced SG-RAG by proposing an additional step called Merging and Ordering Triplets (MOT). The new MOT step seeks to decrease the redundancy in the retrieved triplets by applying hierarchical merging to the retrieved subgraphs. Moreover, it provides an ordering among the triplets using the Breadth-First Search (BFS) traversal algorithm. We conducted experiments on the MetaQA benchmark, which was proposed for multi-hop question-answering in the movies domain. Our experiments showed that SG-RAG MOT provided more accurate answers than Chain-of-Thought and Graph Chain-of-Thought. We also found that merging (up to a certain point) highly overlapping subgraphs and defining an order among the triplets helped the LLM to generate more precise answers.

Keywords: large language model; knowledge graph; question answering; multi-hop question; graph RAG



Academic Editor: Daniel E. O'Leary

Received: 25 May 2025

Revised: 24 July 2025

Accepted: 29 July 2025

Published: 1 August 2025

Citation: Saleh, A.O.M.; Tur, G.; Saygin, Y. SG-RAG MOT: SubGraph Retrieval Augmented Generation with Merging and Ordering Triplets for Knowledge Graph Multi-Hop Question Answering. *Mach. Learn. Knowl. Extr.* **2025**, *7*, 74. <https://doi.org/10.3390/make7030074>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Large language models (LLMs), such as GPT, Gemini, and Llama, have shown a strong ability for natural language understanding and generation. The strong capabilities of LLMs have made them a key component of many solutions regarding natural language processing tasks, especially for generic question answering, where LLMs are capable of generating convincing answers [1]. However, the hallucination problem of these models limits their usage in real scenarios [2]. In the case of domain-specific question answering, hallucinations can be seen as factually wrong, outdated, or irrelevant responses by the LLMs [3]. To decrease the effects of hallucination and help LLMs generate more accurate answers, Retrieval Augmented Generation (RAG) was proposed [4]. The RAG method helps the underlying LLM by embedding a set of relevant textual documents with the given question, and the task of the LLM becomes to extract the answer from this set of documents.

The simplicity of applying the RAG method has made it very common in multiple fields such as Finance [5], Medicine [6], and Religion [7], to name a few. However, the RAG method struggles to resolve hallucination in multi-hop (complex) questions that require the underlying LLM to reason over multiple documents. The semantic similarity-based document retrieval in RAG fails to retrieve the necessary documents to answer multi-hop questions. Recent LLMs, such as Gemini 2.0, have provided longer context windows to enable sharing more knowledge with the model. However, these LLMs have shown that they do not always leverage the entire context provided. The position of the necessary information affects the performance of the underlying LLM; specifically, performance decreases when the required knowledge appears in the middle of a long context [8]. This issue is known in the literature as the lost-in-the-middle problem. In the case of multi-hop questions, answering these questions requires integrating multiple pieces of knowledge. The effect of the lost-in-the-middle problem extends to the distance between the necessary knowledge pieces within the context [9]. One possible solution to this problem is to define an order for the context shared with the LLM, such as ordering the set of paragraphs in the context based on their average attention scores in ascending order [10].

Another attempt to alleviate the limitations of the RAG method is the Graph RAG method [11]. The first appearance of Graph RAG was in a blog by Microsoft Research [12]. The authors in the Microsoft blog suggested converting knowledge from an unstructured textual format into a knowledge graph (KG), where knowledge can be well structured, as a solution to the limitations of RAG for multi-hop questions. Previously, we proposed SubGraph Retrieval Augmented Generation (SG-RAG) as a Graph RAG method for multi-hop knowledge graph question answering [13]. SG-RAG depends on Cypher queries to retrieve the necessary subgraphs from the KG to answer the given questions. Since LLMs expect textual data, SG-RAG transforms the retrieved subgraphs into a textual representation in triplet format. We showed that SG-RAG outperformed RAG for 1-hop, 2-hop, and 3-hop questions using Llama-3 8B Instruct and GPT-4 Turbo as underlying LLMs. Although SG-RAG has shown a high level of performance, the textual transformation step in SG-RAG does not impose an order on the triplets shared with the underlying LLM, leaving us unaware of how the ordering might affect the performance of SG-RAG. We also observed that the shared triplets contain many redundant triplets, which makes the context shared with the LLM longer. Lastly, our previous experiments were limited by Llama-3 8B Instruct being the only open-source LLM we experimented with and by RAG as a baseline.

In order to fill the gap in our previous work and further enhance our method, we propose an extension to the methodology of SG-RAG based on inserting an additional intermediate processing step after the textual transformation step, called Merging and Ordering Triplets (MOT). The goal of MOT is to enhance the efficiency and effectiveness of the knowledge representation within SG-RAG by merging highly overlapping subgraphs, thereby removing redundant triplets and subsequently defining the order among them. We also extended our experiments to cover more open-source LLMs with different sizes and more advanced baselines. The chosen LLMs were Llama-3.1 8B Instruct, Llama-3.2 3B Instruct, Qwen-2.5 7B Instruct, and Qwen-2.5 3B Instruct. Our results showed that SG-RAG MOT outperformed Chain-of-Thought and Graph Chain-of-Thought, which is a state-of-the-art Graph RAG method. Our ablation study on the effect of ordering triplets indicated that defining an order using graph traversal algorithms, such as Breadth-First Search (BFS) and Depth-First Search (DFS), helped the underlying LLM reason about the given triplets and decreased the effect of the lost-in-the-middle problem.

The remainder of this article is structured as follows: Section 2 presents background information and related works. Section 3 provides definitions of the preliminary concepts and a definition of the problem. We present our previous work in Section 4 by explaining the

methodology of SG-RAG and the main findings, and highlighting potential improvements. We describe the Merging and Ordering Triplets (MOT) step as an extension to the SG-RAG method in Section 5. Sections 6 and 7 provide the setup of the experiments and the results, together with the ablation studies and the case studies. Section 8 concludes the article with a summary of the work and highlights future work.

2. Background and Related Work

LLMs have shown significantly high performance in many tasks regarding natural language understanding and generation [1,14]. However, for domain-specific question-answering tasks, LLMs suffer from providing factually wrong, out-of-context, outdated, or irrelevant responses. This problem is known in the literature as hallucination [2]. Because of the large scale of these models, task- or domain-specific fine-tuning becomes challenging. One of the most promising and commonly used solutions is Retrieval Augmented Generation (RAG) [4,15]. The RAG method is based on injecting a set of domain knowledge with the user input and sending them to the LLM. The selection process for the set of knowledge shared by the LLM is based on measuring the semantic similarity between the user input and the entire set of domain knowledge. The top-k most similar piece of knowledge is sent to the LLM. The advantage of using semantic-based knowledge retrieval lies in its ability to perform a broad search over the knowledge base by leveraging semantic relationships between the input question and the available information. This strength is reflected in the performance improvements achieved by RAG for simple questions. However, relying on semantic retrieval often fails to retrieve the necessary pieces of information for answering complex questions that require reasoning across multiple facts [12]

In parallel to domain-specific question answering, the high performance of LLMs has prompted researchers to investigate the potential of LLMs for graph-related tasks [16], such as classifying graph nodes [17]. The potential of using graphs with LLMs has led to the idea of using knowledge graphs with LLMs. Edge et al. [18] worked on answering domain-specific questions at a global level, which requires an understanding and awareness of the domain. To achieve their goal, they used LLMs to transform unstructured domain knowledge into a knowledge graph, and then they divided the knowledge graph into small subgraphs (communities), where each subgraph held summarized information about the subgraph. Since then, the concept of Graph RAG has emerged and been intensively studied as an improvement over the traditional RAG method [11], in which the domain knowledge in Graph RAG is represented as a knowledge graph. Graph Chain-of-Thought (Graph-CoT), proposed by Jin et al. [19], is an example of the Graph RAG method. Graph-CoT tackles the questions that require reasoning by giving the LLM the ability to interact directly with the knowledge graph through a set of predefined functions. For any input question, the method starts from a node that is semantically similar to the input question and then traverses the knowledge graph, collecting the information required to answer the question. Think-on-Graph (ToG), proposed by Sun et al. [20], is another example of a Graph RAG method. Differently from Graph-CoT, ToG traverses the graph using beam search to build multiple reasoning paths, starting from nodes containing the entities that appear in the given question. The task of the LLM in ToG is to score the candidate paths, determine whether the reasoning paths contain enough knowledge to answer the question, and finally generate a response based on the retrieved paths. Both Graph-CoT and ToG are iterative methods in which the underlying LLM plays a major role in each iteration. Therefore, a minor hallucination in an early iteration can dramatically shift the traversal process. This makes the performance of these two methods highly sensitive to the performance and capabilities of the underlying LLM. The most recent example of a Graph RAG method is the

SubGraph Retrieval-Augmented Generation (SG-RAG), which we proposed previously [13]. We discuss SG-RAG in detail later in Section 4.

3. Preliminaries and Problem Definition

In this section, we define the preliminary concepts that we will use in the problem definition and the rest of this paper.

Definition 1 (Graph). A graph is a data structure that consists of a set of nodes denoted by V and a set of edges denoted by E . For each edge $e \in E$, there exist two nodes, $v_i, v_j \in V$, such that e connects v_i and v_j . With respect to E , a graph can be categorized as directed or undirected. The difference between them is that for each edge $e \in E$ in the directed graph, there exist two nodes, $v_i, v_j \in V$, where e connects from v_i to v_j . In the context of the directed graph, we call v_i, v_j the source and destination, respectively.

Definition 2 (Subgraph). Let $G = (V, E)$ and $G' = (V', E')$ be two graphs. We call G' a subgraph of G if and only if $V' \subseteq V$ and $E' \subseteq E$.

Definition 3 (Multigraph). A graph $G = (V, E)$ is called a multigraph if loops and multiple edges connecting the same two vertices are permitted [21]. Similarly to a graph, a multigraph can be either directed or undirected.

Definition 4 (Knowledge Graph). A knowledge graph, denoted by KG , refers to the knowledge stored in a multigraph $G_{KG} = (V_{KG}, E_{KG})$. Each vertex $v \in V_{KG}$ represents an entity, while each edge $e \in E_{KG}$ determines the relationship between the two entities in $v_i, v_j \in V_{KG}$. The nodes and edges are labeled to specify the type of information they carry.

The nodes and edges in KGs may include a set of attributes to embed additional information in a structured format. KG can be hosted in a graph database such as Neo4j (<https://neo4j.com/product/neo4j-graph-database/> accessed on 1 June 2024), where a graph-specific query language can be used to search and retrieve subgraphs. An example of a graph-specific query language is the Cypher query language, developed by Neo4j [22].

Definition 5 (n -hop Question in KG context). Let q be a question expressed in Natural Language, where the answer to q consists of multiple entities in the KG . We call q an n -hop question if answering q requires one or more subgraphs, where each subgraph has n edges. The number of necessary subgraphs is equal to the number of entities in the expected answer to q .

The question “The films staged by Sharon Tate have been released during which years” is an example of a 2-hop question. To find the correct answer to the question, which is “1967, 1968”, you need two subgraphs. Each of these subgraphs has two edges, “STARRED_ACTORS” and “RELEASE_YEAR”.

Problem Definition: Let D be a domain where its knowledge is represented by a knowledge graph KG . The task of domain-specific multi-hop knowledge graph question answering (KGQA) can be defined as generating answers to n -hop questions about D . The generation of the answer is based on the knowledge retrieved from the KG .

4. SubGraph Retrieval Augmented Generation

Subgraph Retrieval Augmented Generation (SG-RAG) is a zero-shot Graph RAG method we proposed in [13] for domain-specific multi-hop KGQA. In this section, we first provide an overview of the SG-RAG method. Then, we highlight the experimental setup and the main findings from our previous work [13]. Lastly, we discuss the potential improvements targeted in our extension.

4.1. Overview of SG-RAG

As Figure 1 shows, the SG-RAG method comprises three main steps: 1. subgraph retrieval, 2. textual transformation, and 3. answer generation. For a user question q , the SG-RAG method begins with the subgraph retrieval step, to retrieve the necessary subgraphs to answer q . The retrieval process is based on mapping q onto a Cypher query c^q (Text2Cypher mapping):

$$c^q = \text{Text2Cypher}(KG, q) \quad (1)$$

such that querying KG using c^q retrieves the set of matched and filtered subgraphs S containing the information necessary to answer q :

$$S = \text{querying}(KG, c^q) \quad (2)$$

where $S = \{G'_1, G'_2, \dots, G'_m\}$. In our initial trials on Text2Cypher mapping, we observed that both LLaMA3-8B and Gemini performed poorly in generating valid Cypher queries. While GPT-4 was able to generate Cypher queries, its accuracy required further improvement. Therefore, we adopted a template-based approach for Text2Cypher mapping, using manually crafted templates. For a given question q , we select a template corresponding to the question type and populate it with the entity mentioned in q . Examples of these templates are provided in Figure A1.

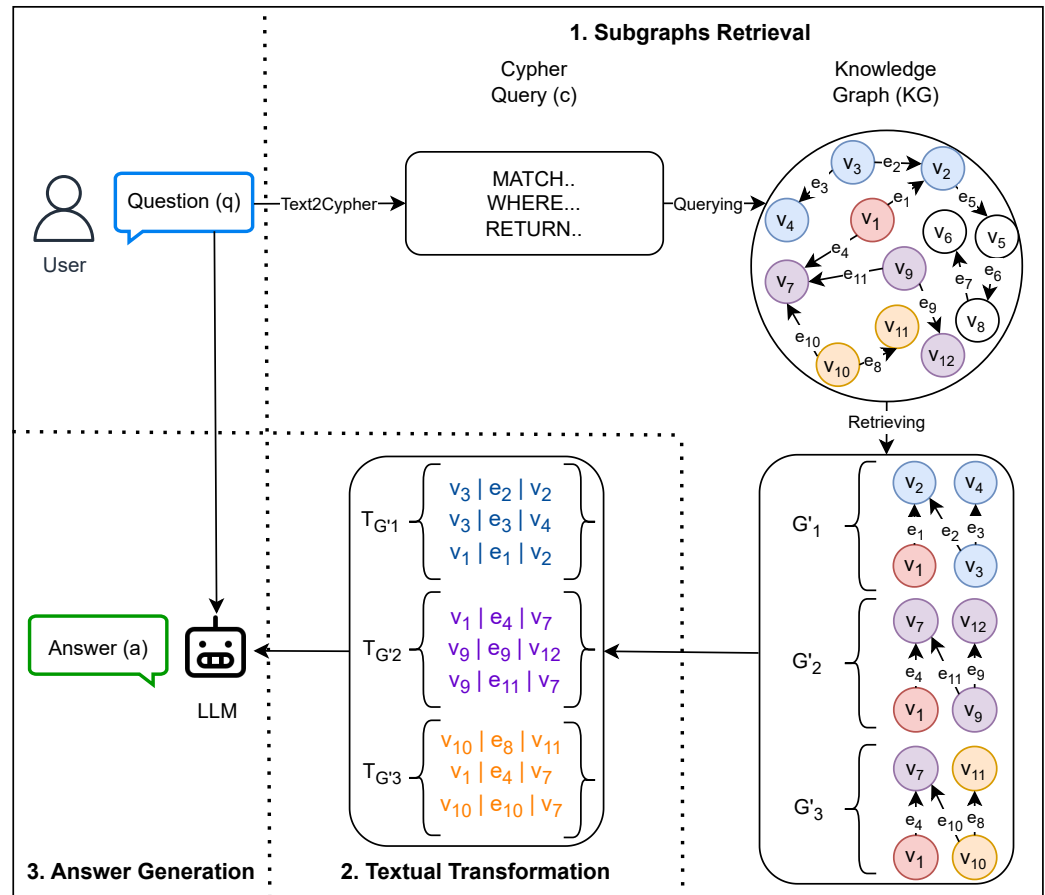


Figure 1. An overview of SG-RAG.

After retrieving the set of subgraphs S , the second step of SG-RAG is to transform the retrieved subgraphs into a textual representation. For each retrieved subgraph $G' = (V', E') \in S$, the transformation is based on converting each directed edge $e \in E'$ with its corresponding source and destination nodes $v_i, v_j \in V'$ into a triplet, denoted by t , in

the form “subject | relation | object”. In the triplet format, “subject” and “object” refer to v_i and v_j , respectively, while “relation” refers to the label e . During the textual transformation, the triplets that belong to the same subgraph are grouped, such that

$$T = \{T_{G'_1}, T_{G'_2}, \dots, T_{G'_m}\} \quad (3)$$

where $T_{G'_i} = \{t_{i1}, t_{i2}, \dots, t_{in}\}$.

Since $T_{G'_i}$ is a set, there is no pre-defined order among the triplets $t_{ij} \in T_{G'_i}$.

Lastly, an answer A to the question q is generated using the underlying LLM as follows:

$$A = \text{LLM}(I, q, T) \quad (4)$$

where I refers to the instructions explaining the task and the inputs to the LLM. The prompt template represented by I is shown in Figure A2.

4.2. Experimental Setup and Main Results

In our previous work [13], we experimented with the MetaQA benchmark dataset proposed by Zhang et al. [23], which is a benchmark for multi-hop KGQA. It contains a KG about the movies domain and a set of question–answer pairs grouped into 1-hop, 2-hop, and 3-hop. The ground truth answers are lists of entities.

As baseline methods to compare with SG-RAG, we employed the following methods:

1. *Direct*: The direct method generates an answer to q based solely on the internal knowledge of the LLM, stored in its weights. This method is important because it tests the degree of knowledge of the underlying LLM for the targeted domain.
2. Retrieval Augmented Generation (RAG) [4]: The work utilized the traditional RAG method, where the external knowledge is a set of textual documents about the targeted domain. The external knowledge is stored in a vector database. Knowledge retrieval is based on the semantic similarity between q and the set of textual documents. The top- k documents most similar to q are sent as a context to the LLM to generate an answer.

The ground-truth answers in the MetaQA benchmark are lists of entity names. Therefore, metrics such as BLEU [24] and ROUGE [25] are not applicable, as they are sensitive to variations in the token order between the ground-truth and generated answers. Moreover, the *Direct* and RAG baselines generate answers either from the internal knowledge of the underlying LLM or from knowledge sources different from MetaQA-KG. As a result, the generated answers may include entities that are not present in the MetaQA ground truth. Penalizing the model for generating such entities could be unfair, as these entities may be correct but outside the scope of MetaQA. For these reasons, we evaluated the performance using the answer-matching rate (AMR) metric, inspired by the entity-matching rate metric proposed by Wen et al. [26] for evaluating dialogue systems. AMR measures the intersection between the ground truth answer A^{gt} and the generated answer A , normalized by the number of entities in A^{gt} :

$$\text{AMR}(q) = \frac{|A^{gt} \cap A|}{|A^{gt}|} \quad (5)$$

AMR is advantageous because it is insensitive to the ordering of entities within the answer. It is recall-focused, penalizing the model for missing entities from the ground-truth answer (via set intersection), but not for generating additional, potentially valid entities outside the MetaQA dataset. The denominator in Equation (5) reflects the number of entities in the ground-truth answer, ensuring that the metric properly captures recall.

To measure the statistical significance of the difference between the performance of our method and the baseline, we applied a one-tailed paired t -test following the guidelines

proposed by Dror et al. [27]. As the size of our test set was large (>30), we could assume that the performance scores would follow a normal distribution [28]. Our null hypothesis (H_0) is that the performance of our method is at most as good as the performance of the baseline. Our alternative hypothesis (H_a) is that our method outperforms the baseline. In the results discussion, we use the term statistically significant when the p -value is less than 0.05.

Initially, we compared the performance of *SG-RAG* with the *Direct* and *RAG* methods; *RAG* used Wikipedia documents regarding the entities in the MetaQA dataset as external knowledge. This experiment was conducted on a test sample of 15K question–answer pairs divided equally among 1-hop, 2-hop, and 3-hop questions. The underlying LLM was the Llama-3 8B Instruct version [29]. From the results shown in Table 1, we can observe that the performance of the *Direct* method was poor compared to the other methods. This shows that depending only on the internal knowledge of Llama-3 8B is insufficient for the domain-specific task. On the other hand, the traditional *RAG* method resulted in improvements in performance for 1-hop and 2-hop questions (18% and 14% increase in 1-hop and 2-hop, respectively, using Top-10) compared to the *Direct* method. However, the performance increase for 3-hop questions was very low (maximum increase was 4% using Top-1), which may have been due to the limitations of the semantic-based retrieval in finding the necessary documents to answer multi-hop questions. This limitations of traditional *RAG* are addressed in *SG-RAG* by utilizing Cypher queries for knowledge retrieval. Although Cypher-based retrieval narrows the search space compared to semantic-based retrieval, its ability to traverse the graph and retrieve subgraphs containing the necessary information gives *SG-RAG* a distinct advantage for multi-hop questions. This advantage is reflected in the statistically significant performance improvements observed for 1-hop, 2-hop, and 3-hop questions (Table 1).

Table 1. Comparison between the performance of *Direct*, *RAG* with Wikipedia documents (*RAG-Wiki*), and *SG-RAG*. We show the performance based on answer-matching rate (AMR). We conducted this experiment using Llama-3.1 8B Instruct.

Method	1-Hop	2-Hop	3-Hop
Direct	0.24	0.13	0.17
RAG-Wiki Top-1	0.33	0.19	0.21
RAG-Wiki Top-2	0.36	0.20	0.20
RAG-Wiki Top-3	0.38	0.22	0.20
RAG-Wiki Top-5	0.40	0.23	0.18
RAG-Wiki Top-10	0.42	0.27	0.19
SG-RAG	0.90	0.73	0.58

One of the possible reasons behind the lower performance of the traditional *RAG* method compared to *SG-RAG* for 1-hop questions is that Wikipedia documents may not contain all the information required to answer a question. To address this issue, we generated textual documents based on the MetaQA-KG using the Gemini model (<https://gemini.google.com/> accessed on 1 July 2024). To generate the documents, we first extracted the entity names from the questions. Then, for each entity, we extracted the node $v \in V_{KG}$ representing the targeted entity and the 1-hop neighborhood surrounding it. Lastly, we converted the extracted subgraph into a set of triplets and sent it to the Gemini model to generate a paragraph regarding the given triplets. During this experiment, we used Gemini 1.5 Flash version. This experiment was conducted on a test sample of 1547 1-hop questions, 1589 2-hop questions, and 1513 3-hop questions. From the results in Table 2, we can see that applying *RAG* to the generated documents based on the *KG* (*RAG-Gen*) increased the

performance for 1-hop questions compared to *RAG-Wiki*. Moreover, we can notice that the performance of *SG-RAG* remained higher than that of *RAG-Gen*, even for 1-hop questions. The reason behind the higher performance of *SG-RAG* is that the knowledge sent to the LLM is in the triplet format, which helps the LLM extract information and apply reasoning to it.

Table 2. Comparison between the performance of RAG on Wikipedia documents (*RAG-Wiki*), RAG on Gemini generated documents (*RAG-Gen*), and *SG-RAG*. We show the performance based on answer-matching rate (AMR). We conducted this experiment using Llama-3.1 8B Instruct.

Method	1-Hop	2-Hop	3-Hop
RAG-Wiki Top-1	0.33	0.19	0.21
RAG-Wiki Top-2	0.35	0.20	0.20
RAG-Wiki Top-3	0.36	0.22	0.20
RAG-Gen Top-1	0.64	0.15	0.17
RAG-Gen Top-2	0.66	0.12	0.13
RAG-Gen Top-3	0.66	0.12	0.16
SG-RAG	0.91	0.72	0.60

Lastly, to analyze the effect of the underlying LLM, we re-applied *SG-RAG* and *RAG-Gen* with GPT-4 Turbo [30] using the same sample we had used previously. As can be seen (Table 3), *SG-RAG* again statistically significantly outperformed the traditional RAG method for 1-hop, 2-hop, and 3-hop questions. We can also see a general increase in the performance of all the methods compared to the results using Llama-3 8B instruct. This increase can be attributed to the higher capabilities of the GPT-4 model compared to Llama-3 8B.

Table 3. Comparison between the performance of RAG on Gemini generated documents (*RAG-Gen*), and *SG-RAG*. We show the performance based on answer-matching rate (AMR). We conducted this experiment with GPT-4 Turbo.

Method	1-Hop	2-Hop	3-Hop
RAG-Gen Top-1	0.765	0.286	0.204
RAG-Gen Top-2	0.776	0.181	0.177
RAG-Gen Top-3	0.784	0.179	0.180
SG-RAG	0.941	0.815	0.520

4.3. Potential Improvements

LLMs generally struggle to pay sufficient attention to knowledge that appears in the middle of the context, and they tend to only focus on knowledge at the beginning and end of the context. This problem in LLMs is known as the lost-in-the-middle problem [8]. For multi-hop QA, the lost-in-the-middle problem increases when the necessary parts of information are far away from each other [9]. A possible solution to this problem is to apply ordering of the knowledge in the context [10,31]. In our previous work, we did not propose an ordering mechanism for the subgraph triplets $T_{G'_i} \in T$; however, specifying the order of the triplets can decrease the effect of the lost-in-the-middle problem by supporting the LLM in reasoning about T and generating a more accurate answer to q .

Moreover, the textual transformation step in *SG-RAG* converts the set of retrieved subgraphs $S = \{G'_1, G'_2, \dots, G'_m\}$ to the set $T = \{T_{G'_1}, T_{G'_2}, \dots, T_{G'_m}\}$ so that each subgraph $G'_i \in S$ is represented by the set of triplets $T_{G'_i} \in T$. In the triplet representation, the subgraphs in T can overlap. This means that the set T that is shared with the LLM to generate an answer to q can contain repetitive triplets. These redundant triplets unnecessarily increase the size of the context shared with the LLM. During our experiments, we noticed

that, on average, 13% of the retrieved triplets for a 2-hop question were redundant. This percentage increased to around 31% in the case of the 3-hop question.

5. Merging and Ordering Triplets

To address the potential improvements discussed in Section 4.3, we introduced an additional step into the SG-RAG method after the textual transformation step, as demonstrated in Figure 2. The new step, called Merging and Ordering Triplets (MOT), is an intermediate processing step that functions to reduce the redundant triplets in T by merging subgraphs that have a high overlap with each other. Subsequently, the MOT step defines the order among the triplets in each subgraph triplet set $T_i \in T$ using a graph traversal algorithm. In the following sub-sections, we explain in detail the processes of Merging Subgraphs (MS) and Ordering Triplets (OT).

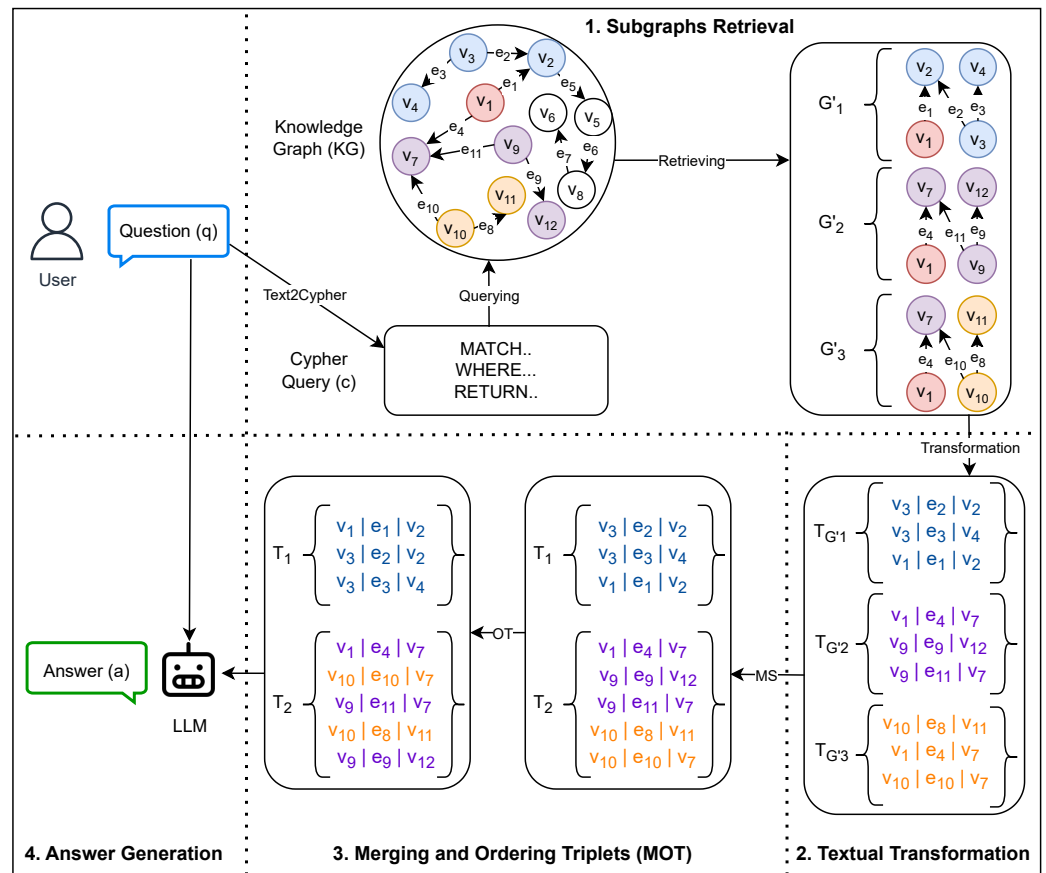


Figure 2. An overview of SG-RAG with merging and ordering triplets extension.

5.1. Merging Subgraphs (MS)

The redundant triplets in T are due to an overlap among the triplet sets in T . Let $T_i, T_j \in T$ such that there exists a triplet t where $t \in T_i$ and $t \in T_j$. Removing t from one of them makes the information in the cropped subgraph incomplete. For that reason, MOT reduces the redundancy by applying agglomerative (bottom-up) hierarchical merging to T , as demonstrated in Algorithm 1. Initially, the hierarchical merging algorithm starts with the set of subgraph triplets T , which is created by the textual transformation step of SG-RAG. In each iteration, it merges the two sets of triplets $T_i, T_j \in T$ such that the overlap between T_i and T_j is the maximum and above the threshold th . This merging forms a new set of triplets $T_k = T_i \cup T_j$. Both T_i and T_j are removed from T and replaced with T_k . The MS algorithm stops when the overlap between any combination of two sets $T_i, T_j \in T$ is below th . Unlike direct graph merging, which would merge all subgraphs into one large graph,

our algorithm iteratively merges only subgraphs with a high level of overlap at the triplet level. This targeted approach allows controlling the degree of merging and redundancy reduction.

Algorithm 1: Merging subgraphs

```

input: Set of Subgraphs' Triplets  $T$ , threshold  $th$ 
repeat
     $maxSim \leftarrow -1$ ;
     $m \leftarrow \text{length}(T)$ ;
    for  $i = 0$  to  $m - 1$  do
        for  $j = i + 1$  to  $m$  do
             $sim \leftarrow \text{jaccard}(T[i], T[j])$ ;
            if  $(sim \geq th) \& (sim > maxSim)$  then
                 $maxI \leftarrow i$ ;
                 $maxJ \leftarrow j$ ;
                 $maxSim \leftarrow sim$ ;
            end
        end
    end
    if  $maxSim \geq th$  then  $T \leftarrow \text{merge}(T, maxI, maxJ)$ ;
until  $maxSim \neq -1$ ;
return  $T$ 
  
```

To measure the overlap between two sets of triplets, we use the Jaccard similarity, as it is a widely used, interpretable metric for measuring set similarity, making it well-suited for comparing triplet sets:

$$\text{Jaccard}(T_i, T_j) = \frac{|T_i \cap T_j|}{|T_i \cup T_j|} \quad (6)$$

The result of the MS is

$$T_{MS} = \{T_1, T_2, \dots, T_l\} \quad (7)$$

where $T_i \in T_{MS}$ is a set of triplets.

The result of the MS depends on the threshold th , which is a hyperparameter. The threshold th can take a value between 0 and 1, where 0 leads to merging all $T_k \in T$ into a single subgraph, while 1 prevents the merging process. This hyperparameter controls the trade-off between decreasing the size of the context shared with the LLM by removing redundant triplets and the performance of the LLM. We discuss the effect of this hyperparameter later in Section 7.2.

5.2. Ordering Triplets (OT)

After completing Merging Subgraphs (MS), Ordering Triplets (OT) orders the triplets in each set of triplets $T_k \in T_{MS}$ independently. Our ordering mechanism uses Breadth-First Search (BFS) as a triplet traversal algorithm. As shown in Algorithm 2, OT requires as inputs the set of triplets $T_k \in T_{MS}$ and the node $v_q \in V_{KG}$, where v_q represents the entity appearing in the question q . OT defines a queue, called *fringe* in Algorithm 2, to hold the nodes that have been reached but not yet explored. The *fringe* initially contains the node v_q . Besides the *fringe* queue, OT also defines an empty list, called *traversed* in Algorithm 2, to carry the traversed triplets. In each iteration, OT obtains the first node v_i from the *fringe* queue, then it retrieves the set of triplets T_s as

$$T_s = \{t = (v_s, e, v_d) \in T_k | v_i = v_s \text{ or } v_i = v_d\} \quad (8)$$

Next, for each triplet $t \in T_s$, t is appended to the *traversed* list if $t \notin traversed$, and the algorithm adds the node v_j , defined as $v_j = v_s$ or $v_j = v_d$ where $v_j \neq v_i$, to the end of the queue *fringe*.

Algorithm 2: BFS ordering triplets

```

input: Set of Triplets  $T_k$ , Node  $v_q$ 
/* Initialize fringe */
fringe  $\leftarrow$  queue();
fringe.enqueue( $v_q$ );
/* Initialize traversed list */
traversed  $\leftarrow$  list();
while fringe.empty() = false do
     $v_i \leftarrow$  fringe.dequeue();
     $T_s \leftarrow$  retrieve_triplets_contain( $T_k, v_i$ );
    foreach  $t \in T_s$  do
        if  $t \notin traversed$  then
            traversed.append( $t$ );
             $v_j \leftarrow$  extract_other_node( $t, v_i$ );
            fringe.enqueue( $v_j$ );
        end
    end
end
return traversed
  
```

Figure 3 provides a zoomed-in visualization of the MOT on the subgraphs G'_2 and G'_3 from Figure 2. We can see that G'_2 and G'_3 are merged into a larger subgraph where the redundant triplet " $v_1 | e_4 | v_7$ " is removed. Next, BFS traverses the merged subgraph.

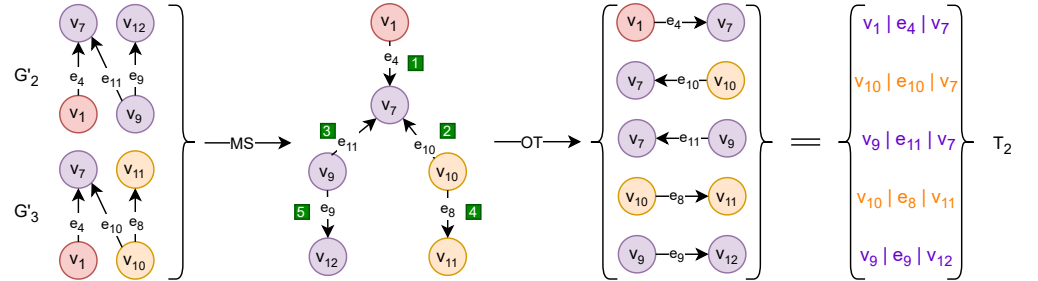


Figure 3. Zoomed-in visualization of the MOT on the subgraphs G'_2 and G'_3 from Figure 2. The numbers in the green squares are the traversing order of the triplets.

6. Experimental Settings

In this section, we provide the settings of our experiments. First, we list the baseline methods we considered. Then, we specify our test set and the evaluation metric we used. Lastly, we discuss the implementation setup for the experiments.

6.1. Baselines

In our evaluation, we compared the performance of *SG-RAG MOT* with the following state-of-the-art methods:

1. *Direct*: Similar to the *Direct* baseline used by Saleh et al. [13]. The prompt used with this method is shown in Figure A3.
2. *Chain-of-Thought (CoT)* [32]: In the CoT method, the LLM answers the question q in a step-by-step approach based on its internal knowledge until it reaches the final answer to q . To give the LLM this ability, we applied a few-shot setup by providing

7 examples as context in the prompt. The prompt template for CoT is illustrated in Figure A4.

3. *Triplet-based RAG*: This method integrates the top- k related knowledge, which is retrieved from the KG in triplet format, with the LLM prompted to generate the answer to q . The retrieval process is based on the semantic similarity between q and the triplets in KG. In our experiments, we applied this method three times, with k being 5, 10, and 20. The prompt used with this method was the same as that used in SG-RAG, which is shown in Figure A2.
4. *Graph Chain-of-Thought (Graph-CoT)* [19]: We applied *Graph-CoT* in the few-shot setup using the implementation of Jin et al. [19], published on GitHub (<https://github.com/PeterGriffinJin/Graph-CoT> accessed on 25 April 2025).

We discarded the Think-on-Graph (ToG) [20] method from our baselines. During our initial experiments, we applied ToG using the implementation of Sun et al. [20] published on GitHub (<https://github.com/GasolSun36/ToG> accessed on 27 April 2025); however, we noticed an incompatibility between ToG and the MetaQA benchmark we used for testing, which led to a low performance for ToG.

6.2. Dataset and Evaluation Metric

Following our previous work [13], we applied our experiments using the MetaQA benchmark dataset proposed by Zhang et al. [23]. All the experiments were conducted on a sample of 3942 question–answer pairs, around 5% of the test set in the MetaQA, and divided equally among number of hops. We hosted MetaQA KG in the Neo4j Graph Database. To evaluate the performance of SG-RAG MOT and the baseline methods, we continued using the answer-matching rate (AMR) metric. We applied a one-tailed paired t -test to measure the statistical significance.

6.3. Experimental Setup

We tested our method and the baselines on multiple open-source LLMs with different sizes. From the Llama family, we chose Llama-3.1 8B Instruct and Llama 3.2 3B Instruct. From the Qwen family, we chose Qwen-2.5 7B Instruct and Qwen-2.5 3B Instruct [33].

The MOT step requires specifying the entity name that appears in the question. To accomplish this, we used a general open-source NER model from Hugging Face (<https://huggingface.co/tomaarsen/span-marker-roberta-large-ontonotes5> accessed on 20 April 2025) that was built based on the Roberta Large model. Since the *Graph-CoT* and the *Triplet-based RAG* methods need to apply semantic-based retrieval as part of their method, we used “all-mpnet-base-v2” (<https://huggingface.co/sentence-transformers/all-mpnet-base-v2> accessed on 27 April 2025) as an embedding model. This model is the default embedding model used by the *Graph-CoT* [19].

Lastly, we ran the methods 3 times for each question, except for the *Graph-CoT*, to decrease the effect of the randomization that can come from the underlying LLMs. For *Graph-CoT*, we executed it only once per question due to the high cost in terms of time and resources, as it requires multiple calls to the LLM.

7. Results and Discussion

In this section, we first highlight the overall performance of our method and the baselines. Then, we discuss the ablation studies, for a further analysis. Lastly, we provide case studies as empirical examples of our method and the baseline, and show some of the failure cases of SG-RAG MOT.

7.1. Overall Performance

The main results are shown in Table 4. From the results, we can observe that (1) *SG-RAG MOT* achieved a statistically significantly higher performance than the other baselines, especially for 2-hop and 3-hop questions. (2) By looking at the performance of *Triplet-based RAG* for 2-hop and 3-hop questions, we can observe that its performance was the lowest. Based on our experiments, we noticed that the triplets retrieved by semantic similarity did not contain the necessary information to answer the 2-hop and 3-hop questions, which misled the underlying LLMs into generating factually wrong answers. As can be seen from the 2-hop example illustrated in Figure 4, the retrieved triplets were either about “Walter Salles”, the entity mentioned in the question, or contained the word “language”, which was also mentioned in the question. The noisy triplets containing “language” were irrelevant to the question, as they were related to the movies written by “Walter Salles”; however, they misled Qwen into generating a factually incorrect answer. (3) For *CoT*, as can be seen from Table 4, it did not provide a performance boost to the *Direct* method for 2-hop and 3-hop questions. We noticed that answering the multi-hop question step-by-step required more knowledge about the targeted domain. (4) The performance of *Graph-CoT* was highly sensitive to the underlying LLM, where its performance changed significantly from one LLM to another. For example, the performance difference between Qwen-2.5 7B Instruct and Llama-3.1 8B Instruct was 33.42%, 42.04%, and 21.05% for 1-hop, 2-hop, and 3-hop questions, respectively. Based on our experiments, we noticed that the iterative approach that the *Graph-CoT* used to traverse KG to reach the answer required a high level of capability from the underlying LLM, where any small hallucination in one iteration led the LLM to hallucinate in all following iterations. (5) The higher performance of Qwen-2.5 7B Instruct for *Triplet-based RAG*, *Graph-CoT*, and *SG-RAG MOT* was due to the stronger prompt engineering capability that we observed during our experiments.

Number of Hops: 2
Question: what are the languages spoken in the films written by Walter Salles
Ground-Truth Answer: Portuguese
Retrieved Triplets
Foreign Land directed_by Walter Salles
Foreign Land written_by Walter Salles
Linha de Passe directed_by Walter Salles
The Motorcycle Diaries directed_by Walter Salles
Behind the Sun directed_by Walter Salles
On the Road directed_by Walter Salles
Behind the Sun written_by Walter Salles
Paris, je t'aime written_by Walter Salles
Going Hollywood in_language French
Dark Water directed_by Walter Salles
Arabian Nights in_language Italian
Goodbye to Language has_imdb_rating good
Les Misérables in_language German
Life of Pi in_language French
A Film Unfinished in_language German
Inglourious Basterds in_language French
Paris, je t'aime directed_by Walter Salles
The Intouchables in_language French
Les Misérables in_language French
Les Misérables in_language French
Qwen-2.5 7B Response: French, Italian, German
Answer is INCORRECT

Figure 4. Example demonstrated the weakness of *Triplet-based RAG* for 2-hop questions.

Table 4. Comparison between the performance of *Direct*, *CoT*, *Triplet-based RAG*, *Graph-CoT*, and *SG-RAG MOT*. We show performance based on answer-matching rate (AMR) as a percentage. For *SG-RAG MOT*, we set the merging threshold th as 0.4 and used Llama-3.1 8B Instruct as the underlying LLM. Bold values indicate the best performance in each column.

Method	LLM	1-Hop	2-Hop	3-Hop
Direct	Llama-3.1 8B Instruct	36.43	22.77	18.01
	Llama-3.2 3B Instruct	21.30	12.13	8.72
	Qwen-2.5 7B Instruct	16.46	18.61	15.14
	Qwen-2.5 3B Instruct	11.85	12.96	8.05
CoT	Llama-3.1 8B Instruct	39.27	21.81	14.25
	Llama-3.2 3B Instruct	23.01	13.95	13.38
	Qwen-2.5 7B Instruct	18.45	18.99	15.62
	Qwen-2.5 3B Instruct	13.08	14.36	8.79
Triplet RAG Top 5	Llama-3.1 8B Instruct	54.94	4.58	9.85
	Llama-3.2 3B Instruct	52.24	5.27	12.83
	Qwen-2.5 7B Instruct	56.73	5.91	11.68
	Qwen-2.5 3B Instruct	53.60	3.71	12.13
Triplet RAG Top 10	Llama-3.1 8B Instruct	60.28	6.06	12.66
	Llama-3.2 3B Instruct	58.25	6.42	15.23
	Qwen-2.5 7B Instruct	61.82	6.53	13.20
	Qwen-2.5 3B Instruct	57.31	4.27	13.87
Triplet RAG Top 20	Llama-3.1 8B Instruct	63.87	7.18	14.63
	Llama-3.2 3B Instruct	61.46	7.12	16.79
	Qwen-2.5 7B Instruct	64.68	6.75	14.14
	Qwen-2.5 3B Instruct	58.55	5.58	14.62
Graph-CoT	Llama-3.1 8B Instruct	47.98	15.38	4.33
	Llama-3.2 3B Instruct	25.11	9.92	6.41
	Qwen-2.5 7B Instruct	81.40	57.42	25.35
	Qwen-2.5 3B Instruct	51.83	13.65	6.48
SG-RAG MOT		85.26	77.27	65.63

7.2. Ablation Study

How do different open-source LLMs with different sizes perform for SG-RAG MOT? In the main results, we showed the performance of *SG-RAG MOT* using Llama-3.1 8B Instruct. In this experiment, we wanted to explore its performance using different open-source LLMs with different sizes. The results for *SG-RAG MOT* with different underlying LLMs are shown in Table 5. From the results, we can see that the performance of *SG-RAG MOT* changed with the different underlying LLMs. However, the performance was not related to the size of the model, as we can see that the best performance was achieved by Qwen-2.5 7B Instruct, which was higher than Llama-3.1 8B Instruct by 3.54%, 9.25%, and 2.87% for 1-hop, 2-hop, and 3-hop, respectively. Moreover, the performances of Llama-3.1 8B and Llama-3.2 3B were similar to each other for 2-hop and 3-hop questions. Our results indicate that LLMs with a higher ability to reason and extract answers from the given subgraphs triplets can achieve a higher performance for *SG-RAG MOT*.

Table 5. Comparison between the performance of different open-source LLMs for *SG-RAG MOT*. We show the performance based on answer-matching rate (AMR) as a percentage. We set the merging threshold th as 0.4. Bold values indicate the best performance in each column.

LLM	1-Hop	2-Hop	3-Hop
Llama-3.1 8B Instruct	85.26	77.27	65.63
Llama-3.2 3B Instruct	72.62	77.43	65.75
Qwen-2.5 7B Instruct	88.80	86.52	68.50
Qwen-2.5 3B Instruct	81.40	75.25	57.75

What is the effect of the Breadth-First Search (BFS) traversal algorithm on the performance of SG-RAG MOT? To answer this question, we re-applied SG-RAG MOT with the merging threshold th set as 0 and used different ordering strategies. We applied this experiment with $th = 0$, to have only one large subgraph where the effect of the ordering could reach the maximum. The ordering strategies we tested were as follows: (1) Breadth-First Search (BFS) traversal algorithm, which we used as the main ordering strategy, (2) the reverse of BFS, which reverses the order defined by BFS, (3) Depth-First Search (DFS) traversal algorithm, (4) the reverse of DFS, and (5) Random ordering following the standard SG-RAG [13]. The results of SG-RAG MOT with the selected ordering strategies are shown in Table 6. From the results, we can see that the performance of the Random ordering was lower than the performance of BFS and DFS. Moreover, when we look at the performance of the reverse BFS, we can see that it was lower than the performance of BFS and closer to the performance of the Random ordering. We can observe the same for the performance of reverse DFS. This result indicates that defining an order among the triplets in a subgraph helps LLMs to reason over the given subgraph and extract the right answer, which leads to a decrease in the hallucination problem. The advantage of BFS and DFS over their reverse versions is that the first (top) triplets in the BFS and DFS ordering contain the entity appearing in the question q , and the following triplets are connected to the previous one, which helps the LLMs to concentrate on all given triplets, until the last triplet. Although our results (Table 6) did not indicate any statistically significant difference between the performance of BFS and DFS, DFS explores a graph more deeply, making it more susceptible to issues such as potential loops and needing stronger reasoning capabilities to correctly handle backtracking. Therefore, further experiments on different KGs with varying levels of multi-hop question difficulty are needed to investigate the robustness of BFS and DFS.

Table 6. Comparison among different ordering strategies for SG-RAG MOT with a merging threshold $th = 0$.

LLM	Ordering Strategy	2-Hop	3-Hop
Llama-3.1 8B Instruct	Random	67.69	45.81
	DFS	73.24	50.98
	Reverse DFS	68.82	46.68
	BFS	72.64	51.59
	Reverse BFS	69.39	50.37
Llama-3.2 3B Instruct	Random	69.75	42.93
	DFS	73.98	46.52
	Reverse DFS	69.44	43.60
	BFS	74.78	46.29
	Reverse BFS	69.65	43.34
Qwen-2.5 7B Instruct	Random	77.07	43.81
	DFS	82.58	50.45
	Reverse DFS	80.98	45.98
	BFS	81.88	48.23
	Reverse BFS	79.76	49.36
Qwen-2.5 3B Instruct	Random	65.12	36.98
	DFS	69.47	41.25
	Reverse DFS	67.77	38.42
	BFS	70.49	42.42
	Reverse BFS	67.28	38.26

What is the importance of the Merging Threshold th in the performance of SG-RAG MOT? To answer this question, we tested SG-RAG MOT with different values of merging

threshold th . The selected values ranged from 0 to 0.5 and 1. We did not test the values ranged from 0.55 to 0.95, as no further triplet reduction occurred in that range, making those thresholds effectively equivalent to $th = 1$. When th was set to 0, all subgraphs were merged to create one subgraph, which means that all redundant triplets were removed. As the value of th increased, we had more subgraphs and more redundant triplets, until th reached 1. When th was 1, the subgraphs remained as they were without merging and without removing redundant triplets. Figure 5 shows the performance of SG-RAG MOT with the different values of th for all the questions in our test set. We can observe from the results that the performance increased as the value of th increased to a specific value, which was 0.4 in our case. After that value, the performance did not change significantly. This observation highlights the importance of the merging threshold th in controlling the trade-off between the performance of SG-RAG MOT and the redundancy in the triplets. Decreasing the value of th close to 0 leads to a decrease in the duplicated triplets, which makes the context shared with the underlying LLM shorter and helps the LLM to infer faster; however, this comes with a sacrifice in the performance of the method; in our case, there was a performance decrease of 7.5% on average compared to the peak performance. We can take a deeper look in Figure 6, which shows the performance of the SG-RAG MOT for the 3-hop questions only. The percentage enclosed by parentheses on the x-axis shows the percentage of reduction in the number of triplets. We can see that when th was 0.4, we decreased the number of triplets by 12.74% and obtained high performance with the different underlying LLMs. As th is a hyperparameter that controls the trade-off between reducing triplet redundancy and maintaining performance, the optimal value of th can vary across different domains or datasets, depending on factors such as the density and redundancy of knowledge in the graphs, as well as the reasoning capabilities of the underlying LLM. For example, domains with more repetitive knowledge may benefit from lower thresholds (more merging), while domains with diverse, fine-grained knowledge may require higher thresholds (less merging) to avoid losing important distinctions.

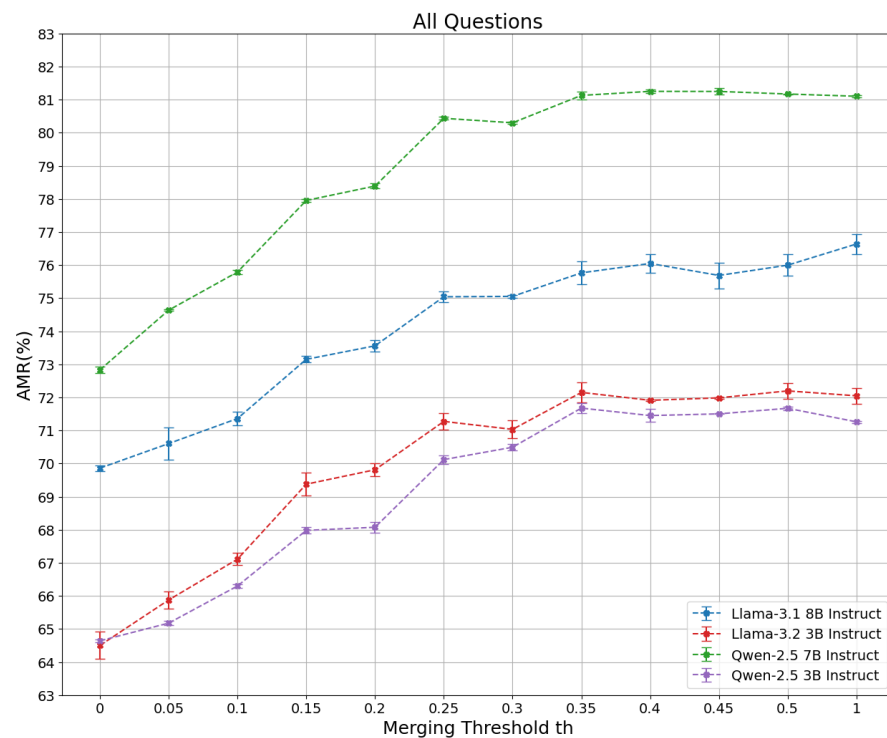


Figure 5. The performance of SG-RAG MOT with different merging threshold values for all questions.

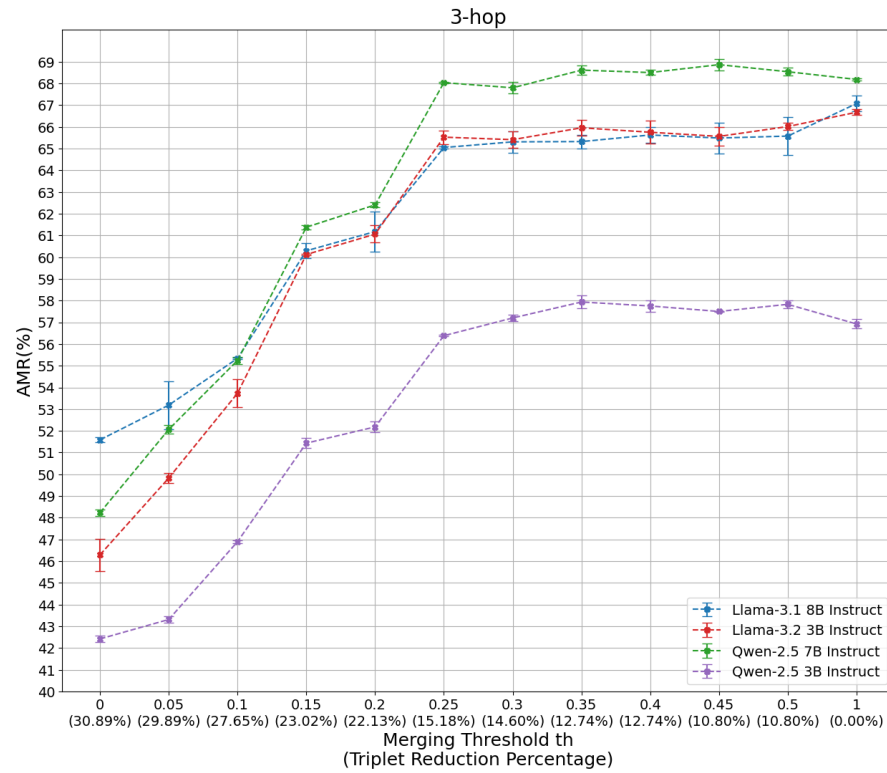


Figure 6. The performance of SG-RAG MOT with different merging threshold values for 3-hop questions.

7.3. Case Studies

We conducted case studies for two main reasons.: (1) To provide an empirical example of the responses generated with SG-RAG MOT and the other baselines. (2) To highlight the weaknesses of SG-RAG MOT. In Figure 7, we can see an example of a 3-hop question from MetaQA. The question given in this example is 3-hop because it requires identifying the director of the “Song of the Exile” movie, then finding movies that were directed by the director of “Song of the Exile”, and finally finding the names of the actors and actresses who appeared in these movies. From SG-RAG MOT, we can see that the Cypher query searched for the required knowledge in the KG and retrieved it as triplets, which led the LLM to generate a correct and precise answer. For Graph-CoT, one of the names in the final answer was correct, while the rest were wrong. This may have been due to the underlying LLM controlling the traversal process of the knowledge graph, where a small hallucination from the underlying LLM can misdirect the the direction of the traversal. In the case of the CoT and Triplet-based RAG methods, both failed to generate correct answers.

During our case studies, we observed three types of failure cases. The first type was related to the language quality of the input questions, as illustrated in Figure 8. With the MetaQA benchmark [23], the questions were divided into two groups based on how they were generated. The first group, called “Vanilla”, consisted of questions created through templates. The second group, called “NTM”, included questions generated by paraphrasing the Vanilla questions. The paraphrasing process involved translating the questions into French and then back into English, which introduced language corruption into some questions, making them difficult to understand, as seen in Figure 8b. Table 7 shows that the performance of SG-RAG MOT dropped significantly for NTM questions compared to Vanilla questions when using different LLMs. Applying a two-tailed *t*-test reveals that the *p*-values were extremely small ($p < 0.05$), leading us to conclude that the performance of our method was highly affected by the language quality of the questions.

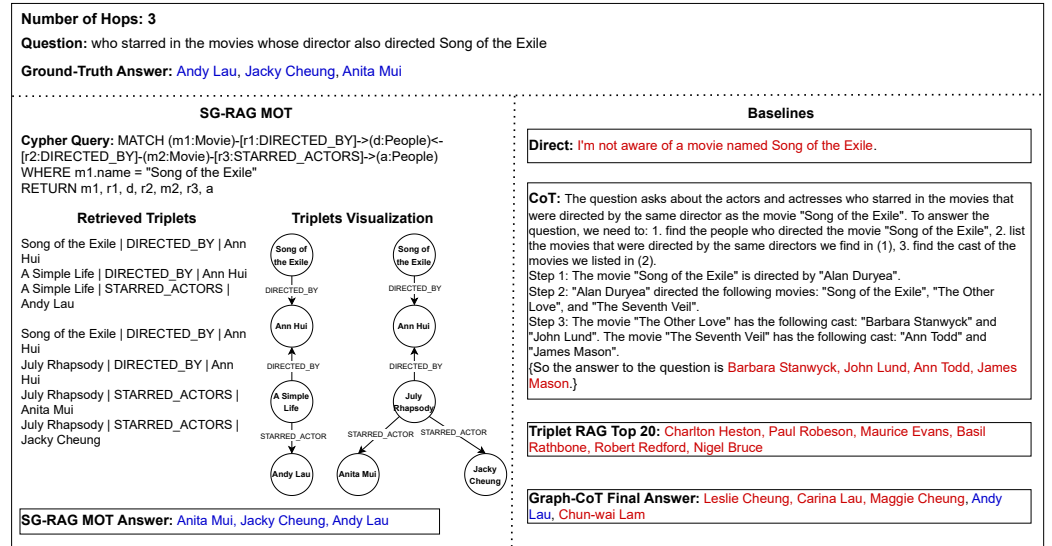


Figure 7. Example of responses generated with SG-RAG MOT and the other baselines for a 3-hop question.

Table 7. Comparison between the performance of SG-RAG MOT on Vanilla and NTM questions. We show the performance based on answer-matching rate (AMR) as a percentage. We set the merging threshold th as 0.4.

LLM	Vanilla	NTM	p -Value
Llama-3.1 8B Instruct	82.39	69.70	2.85×10^{-29}
Llama-3.2 3B Instruct	77.68	66.15	3.28×10^{-24}
Qwen-2.5 7B Instruct	87.50	75.00	7.73×10^{-30}
Qwen-2.5 3B Instruct	76.73	66.17	3.28×10^{-24}

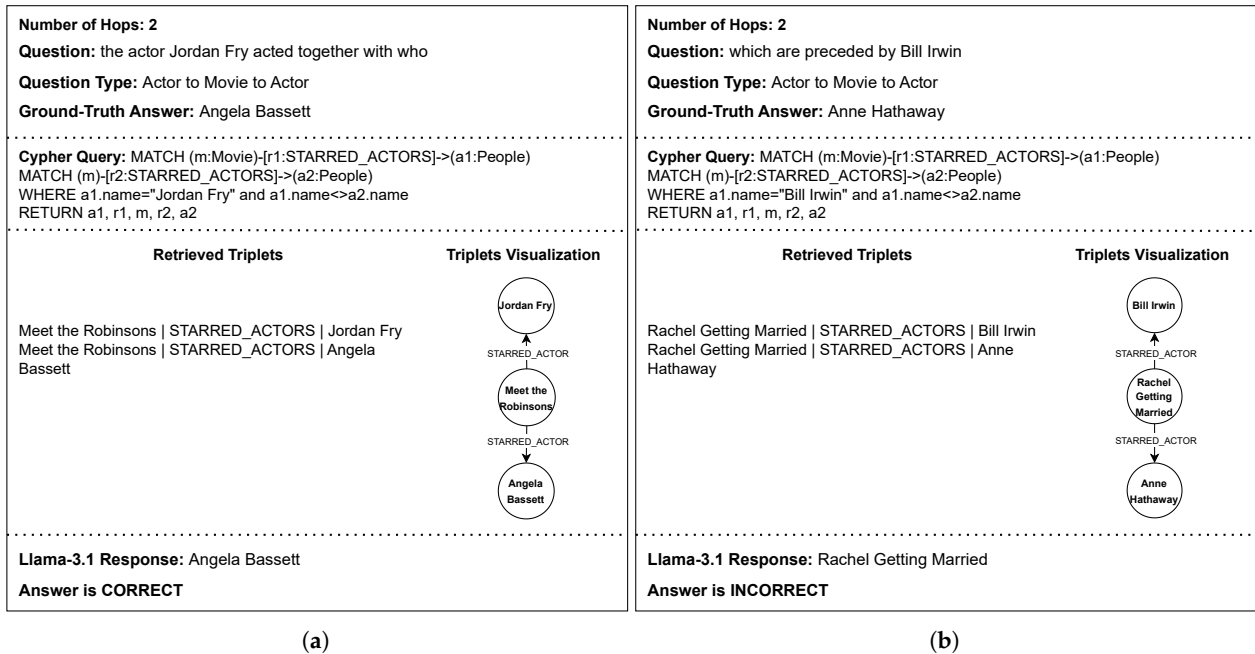


Figure 8. The effect of the language quality of the question on the performance of SG-RAG MOT: (a) successful case, because of good language quality; (b) failure case, because of bad language quality.

The second type of failure case is exemplified in Figure 9a, which shows a 2-hop question where the entity “Brad Bird” is repeated nine times in the retrieved triplets. This high repetition confused the underlying LLM and misled it into selecting the most

repeated entity as the correct answer. To further analyze this observation, we measured the linear correlation, using the Pearson correlation coefficient, between the number of entity repetitions and the performance of our method. This experiment was conducted using Vanilla questions to isolate the effect of language quality. Table 8 shows that the Pearson coefficients ranged from -0.30 to -0.49 across the different underlying LLMs, indicating a moderate negative linear relationship between the number of entity repetitions and our method's performance. However, the degree of this negative relationship varied depending on the underlying LLM. This type of entity-level repetition is not addressed by the MOT step, as our method focuses on redundancy at the triplet level.

Table 8. Measuring the linear correlation between the number of entity repetitions and the performance of SG-RAG MOT using Pearson coefficient. We set the merging threshold th as 0.4.

LLM	n-Hop	Pearson	p -Value
Llama-3.1 8B Instruct	2-hop	-0.3049	1.82×10^{-15}
	3-hop	-0.3084	5.28×10^{-16}
Llama-3.2 3B Instruct	2-hop	-0.4951	1.47×10^{-41}
	3-hop	-0.3221	2.16×10^{-17}
Qwen-2.5 7B Instruct	2-hop	-0.4108	6.87×10^{-28}
	3-hop	-0.3607	1.05×10^{-21}
Qwen-2.5 3B Instruct	2-hop	-0.4454	4.87×10^{-33}
	3-hop	-0.4543	6.31×10^{-35}

The third type of failure case can be seen in the example on the right side of Figure 9, where the number of retrieved triplets is high. In this case, 76 triplets were retrieved, but they were cropped in the figure due to space limitations. Although the triplets contained the necessary knowledge to answer the question fully and precisely, the long context prevented the LLM from effectively leveraging all the provided knowledge. To analyze this observation, we measured the linear correlation, using the Pearson correlation coefficient, between the number of retrieved triplets in the context and the performance of our method. As in the previous experiment, we used Vanilla questions to exclude the effect of language quality. The Pearson coefficients presented in Table 9 indicate a moderate negative linear relationship between the number of retrieved triplets and our method's performance. However, the degree of this negative relationship varied based on the underlying LLM. In Section 8, we discuss potential candidate solutions to overcome these limitations as future work.

Table 9. Measuring the linear correlation between the number of retrieved triplets in the context and the performance of SG-RAG MOT using Pearson coefficient. We set the merging threshold th as 0.4.

LLM	n-Hop	Pearson	p -Value
Llama-3.1 8B Instruct	2-hop	-0.3155	1.63×10^{-16}
	3-hop	-0.3417	1.64×10^{-19}
Llama-3.2 3B Instruct	2-hop	-0.4905	1.03×10^{-40}
	3-hop	-0.3471	4.07×10^{-20}
Qwen-2.5 7B Instruct	2-hop	-0.4146	1.96×10^{-28}
	3-hop	-0.3697	8.45×10^{-23}
Qwen-2.5 3B Instruct	2-hop	-0.4578	4.88×10^{-35}
	3-hop	-0.4479	6.97×10^{-34}

Number of Hops: 2 Question: which person directed the films written by Brad Bird Cypher Query: MATCH (w:People)-[r1:WRITTEN_BY]-(m:Movie)-[r2:DIRECTED_BY]-(d:People) WHERE w.name = "Brad Bird" RETURN w, r1, m, r2, d Retrieved Triplets The Incredibles WRITTEN_BY Brad Bird The Incredibles DIRECTED_BY Brad Bird The Iron Giant DIRECTED_BY Brad Bird The Iron Giant WRITTEN_BY Brad Bird Ratatouille WRITTEN_BY Brad Bird Ratatouille DIRECTED_BY Brad Bird Ratatouille WRITTEN_BY Brad Bird Ratatouille DIRECTED_BY Jan Pinkava Jack-Jack Attack WRITTEN_BY Brad Bird Jack-Jack Attack DIRECTED_BY Brad Bird Llama-3.1 Response: Brad Bird Answer is INCOMPLETE	Number of Hops: 3 Question: what are the genres of the movies whose writers also wrote Apocalypse Now Cypher Query: MATCH (m1:Movie)-[r1:WRITTEN_BY]-(w:People) MATCH (m2:Movie)-[r2:WRITTEN_BY]-(w) MATCH (m2)-[r3:HAS_GENRE]-(g:Genre) WHERE m1.name = "Apocalypse Now" RETURN m1, r1, w, r2, m2, r3, g Part of the Retrieved Triplets Apocalypse Now WRITTEN_BY Francis Ford Coppola The Rainmaker WRITTEN_BY Francis Ford Coppola The Rainmaker HAS_GENRE Drama Apocalypse Now WRITTEN_BY Francis Ford Coppola The Godfather WRITTEN_BY Francis Ford Coppola The Godfather HAS_GENRE Crime Apocalypse Now WRITTEN_BY Francis Ford Coppola Tetro WRITTEN_BY Francis Ford Coppola Tetro HAS_GENRE Drama Apocalypse Now WRITTEN_BY John Milius Red Dawn WRITTEN_BY John Milius Red Dawn HAS_GENRE War Apocalypse Now WRITTEN_BY John Milius Extreme Prejudice WRITTEN_BY John Milius Extreme Prejudice HAS_GENRE Action Apocalypse Now WRITTEN_BY John Milius Jeremiah Johnson WRITTEN_BY John Milius Jeremiah Johnson HAS_GENRE Western Apocalypse Now WRITTEN_BY Francis Ford Coppola Youth Without Youth WRITTEN_BY Francis Ford Coppola Youth Without Youth HAS_GENRE Fantasy Apocalypse Now WRITTEN_BY Francis Ford Coppola Dementia 13 WRITTEN_BY Francis Ford Coppola Dementia 13 HAS_GENRE Horror Apocalypse Now WRITTEN_BY Francis Ford Coppola Patton WRITTEN_BY Francis Ford Coppola Patton HAS_GENRE War Patton HAS_GENRE Biography Apocalypse Now WRITTEN_BY Francis Ford Coppola The Cotton Club WRITTEN_BY Francis Ford Coppola The Cotton Club HAS_GENRE Crime The Cotton Club HAS_GENRE Drama Apocalypse Now WRITTEN_BY Francis Ford Coppola Twixt WRITTEN_BY Francis Ford Coppola Twixt HAS_GENRE Thriller Twixt HAS_GENRE Horror Llama-3.1 Response: War, Drama Answer is INCOMPLETE
--	--

(a)

(b)

Figure 9. Failure cases of *SG-RAG MOT*: (a) failed because of high repetition of “Brad Bird” entity in the retrieved triplets; (b) failed because of high number of retrieved triplets.

8. Conclusions and Future Work

In our previous work, we proposed SubGraph Retrieval Augmented Generation (*SG-RAG*) as a Graph RAG method for multi-hop knowledge graph question answering [13]. We also showed that *SG-RAG* outperformed the traditional *RAG* method, making it a promising solution for many domains, such as medicine, finance, or any other field where the domain knowledge is represented as a knowledge graph. In this work, we aimed to further enhance the performance of *SG-RAG* by introducing a new step called Merging and Ordering Triplets (*MOT*). The *MOT* method is an intermediate processing step that focuses on enhancing the efficiency and effectiveness of knowledge representation. It seeks to decrease the redundancy in the retrieved subgraph triplets by applying hierarchical merging. It also defines an order among the triplets using the Breadth-First Search traversal algorithm. Although we presented *MOT* as an optimization for *SG-RAG*, it is not exclusive to it. This method can also be integrated with other Graph RAG frameworks that retrieve knowledge in the form of subgraphs represented by triplets. We conducted our experiments using four different open-source LLMs. The *SG-RAG MOT* performed significantly better than Chain of Thought (*CoT*), *Triplet-based RAG*, and Graph Chain-of-Thoughts (*Graph-CoT*) with the chosen LLMs. During our ablation studies, we found that the performance of our method was not related to the size of the model, such that an LLM with a higher ability to reason and extract answers from the given subgraphs triplets can achieve a higher performance with *SG-RAG MOT*. Moreover, our results showed that using a graph traversal algorithm, such as Breadth-First Search and Depth-First Search, helped the LLM with reasoning about the given triplets and decreased the effect of the lost-in-the-middle problem. Lastly, our ablation studies highlighted the trade-off between reducing triplet redundancy and maintaining performance, which is controlled by the merging threshold *th*. However, the optimal value of *th* can vary across different domains and datasets.

Although we achieved good performance on the MetaQA benchmark, there is still room for improvement. Starting with the weaknesses described in our case studies, fine-tuning the underlying LLM on contexts presented in the triplet format in a domain-agnostic manner could alleviate these issues. In this work, we evaluated our method on a single do-

main, which was the movies domain, using the MetaQA benchmark. However, validating our method across a broader range of datasets could enhance its generalizability and robustness. One of the main limitations was the use of template-based Text2Cypher mapping. To address this, we plan to automate the Text2Cypher mapping by fine-tuning an LLM such as LLAMA3-8B, enabling it to produce Cypher queries given a question and the corresponding graph schema. Once we complete our work on automated Text2Cypher mapping, we aim to conduct a comprehensive empirical study evaluating the entire pipeline on diverse multi-hop question-answering datasets. The study will allow us to further investigate the robustness of BFS as a triplet ordering technique compared to DFS. It will also help highlight the effect of the merging threshold th across different knowledge graphs.

On the other hand, the reliance of SG-RAG MOT on Cypher queries to retrieve relevant subgraphs limits the flexibility of our method, as the retrieval heavily depends on entities explicitly mentioned in the input question. To overcome this limitation, we plan to develop a hybrid approach that combines the flexibility of the standard RAG method with the multi-hop reasoning capabilities of SG-RAG. Our strategy involves using semantic-based search to identify entities semantically related to those in the input question, followed by Cypher-based retrieval using the identified entities. Lastly, the knowledge graphs in real applications can be noisy, for instance, due to the repetition of entities. Such noise in the KG can degrade the performance of our method. Addressing this limitation could be another potential direction for future work.

Author Contributions: Conceptualization, A.O.M.S., G.T. and Y.S.; Formal analysis, A.O.M.S., G.T. and Y.S.; Investigation, A.O.M.S.; Methodology, A.O.M.S.; Software, A.O.M.S.; Supervision, G.T. and Y.S.; Validation, A.O.M.S., G.T. and Y.S.; Visualization, A.O.M.S.; Writing—original draft, A.O.M.S.; Writing—review and editing, Y.S. All authors will be updated at each stage of manuscript processing, including submission, revision, and revision reminder, via emails from our system or the assigned Assistant Editor. All authors have read and agreed to the published version of the manuscript.

Funding: This research was partially funded by the Scientific and Technological Research Council of Turkey (TUBITAK)’s Industrial PhD program under project number 118C056.

Data Availability Statement: The data presented in this article were obtained from a third party and are available on the GitHub repository <https://github.com/yuyuz/MetaQA> (accessed on 25 May 2024).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this article:

LLM	Large Language Model
KG	Knowledge Graph
RAG	Retrieval Augmented Generation
SG-RAG	SubGraph Retrieval Augmented Generation
MOT	Merging and Ordering Triplets
MS	Merging Subgraph
OT	Ordering Triplets
BFS	Breadth-First Search
DFS	Depth-First Search
CoT	Chain-of-Thought
ToG	Think-on-Graph

Appendix A. Cypher Query Templates

1-hop	Movie to Language	MATCH (m:Movie)-[r:IN_LANGUAGE]->(l:Language) WHERE m.name=<entity> RETURN m, r, l
	Director to Movie	MATCH (m:Movie)-[r:DIRECTED_BY]->(d:People) WHERE d.name=<entity> RETURN m, r, d
2-hop	Writer to Movie to Genre	MATCH (w:People)-[r1:WRITTEN_BY]-(m:Movie)-[r2:HAS_GENRE]->(g:Genre) WHERE w.name=<entity> RETURN w, r1, m, r2, g
	Actor to Movie to Year	MATCH (a:People)-[r1:STARRED_ACTORS]-(m:Movie)-[r2:RELEASE_YEAR]->(y:Year) WHERE a.name=<entity> RETURN a, r1, m, r2, y
3-hop	Movie to Director to Movie to Actor	MATCH (m1:Movie)-[r1:DIRECTED_BY]->(d:People)-[r2:DIRECTED_BY]-(m2:Movie)-[r3:STARRED_ACTORS]->(a:People) WHERE m1.name=<entity> RETURN m1, r1, d, r2, m2, r3, a
	Movie to Writer to Movie to Language	MATCH (m1:Movie)-[r1:WRITTEN_BY]->(w:People)-[r2:WRITTEN_BY]-(m2:Movie)-[r3:IN_LANGUAGE]->(l:Language) WHERE m1.name=<entity> RETURN m1, r1, w, r2, m2, r3, l

Figure A1. Example of our Cypher query templates for MetaQA KG.

Appendix B. Prompt Templates

System Instruction	You are a helpful assistant. Your role is to answer the user's question based on the knowledge provided to you as context.
Prompt	Please answer my question provided below based on the knowledge provided as context. The knowledge in the context is a set of subgraphs represented as triplets. Each triplet is formatted as "subject relation object". Your answer should be accurate and based on the knowledge in the context. Don't generate any further explanation rather than the answer. The Context: <triplets> My question: <question>

Figure A2. The prompt template used with SG-RAG, SG-RAG MOT, and Triplet-based RAG.

System Instruction	You are a helpful assistant. Your role is to answer the user's question precisely.
Prompt	Please answer my question provided below. Your answer should be precise and accurate. Don't generate any further explanation other than the answer. My question: <question>

Figure A3. The prompt template used with the *Direct Method*.

System Instruction	You are a helpful assistant. Your role is to answer the user's question following the Chain-of-Thought approach.
Prompt	Please answer my question provided below. Your final answer should be accurate, enclosed by curly parentheses "{}", and based on a Chain-of-Thought approach as demonstrated in the examples. Here are some examples: <few_shot_examples> (END OF EXAMPLES) My question: <question>

Figure A4. The prompt template used with the Chain-of-Thought (CoT) method.

References

1. Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv* **2023**, arXiv:2307.09288. [\[CrossRef\]](#)
2. Tonmoy, S.; Zaman, S.; Jain, V.; Rani, A.; Rawte, V.; Chadha, A.; Das, A. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv* **2024**, arXiv:2401.01313. [\[CrossRef\]](#)
3. Li, J.; Chen, J.; Ren, R.; Cheng, X.; Zhao, W.X.; Nie, J.Y.; Wen, J.R. The dawn after the dark: An empirical study on factuality hallucination in large language models. *arXiv* **2024**, arXiv:2401.03205. [\[CrossRef\]](#)
4. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.t.; Rocktäschel, T.; et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 9459–9474.
5. Setty, S.; Jijo, K.; Chung, E.; Vidra, N. Improving Retrieval for RAG based Question Answering Models on Financial Documents. *arXiv* **2024**, arXiv:2404.07221. [\[CrossRef\]](#)
6. Zakka, C.; Shad, R.; Chaurasia, A.; Dalal, A.R.; Kim, J.L.; Moor, M.; Fong, R.; Phillips, C.; Alexander, K.; Ashley, E.; et al. Almanac—Retrieval-augmented language models for clinical medicine. *Nejm AI* **2024**, *1*, AIoa2300068. [\[CrossRef\]](#)
7. Alan, A.Y.; Karaarslan, E.; Aydin, O. A RAG-based Question Answering System Proposal for Understanding Islam: MufassirQAS LLM. *arXiv* **2024**, arXiv:2401.15378. [\[CrossRef\]](#)
8. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; Liang, P. Lost in the middle: How language models use long contexts. *arXiv* **2023**, arXiv:2307.03172. [\[CrossRef\]](#)
9. Baker, G.A.; Raut, A.; Shaier, S.; Hunter, L.E.; von der Wense, K. Lost in the Middle, and In-Between: Enhancing Language Models' Ability to Reason Over Long Contexts in Multi-Hop QA. *arXiv* **2024**, arXiv:2412.10079. [\[CrossRef\]](#)
10. Peysakhovich, A.; Lerer, A. Attention sorting combats recency bias in long context language models. *arXiv* **2023**, arXiv:2310.01427. [\[CrossRef\]](#)
11. Zhang, Q.; Chen, S.; Bei, Y.; Yuan, Z.; Zhou, H.; Hong, Z.; Dong, J.; Chen, H.; Chang, Y.; Huang, X. A Survey of Graph Retrieval-Augmented Generation for Customized Large Language Models. *arXiv* **2025**, arXiv:2501.13958.

12. Larson, J.; Truitt, S. GraphRAG: Unlocking LLM Discovery on Narrative Private Data. 2024. Available online: <https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/> (accessed on 25 June 2024).
13. Saleh, A.O.; Tür, G.; Saygin, Y. SG-RAG: Multi-Hop Question Answering With Large Language Models Through Knowledge Graphs. In Proceedings of the 7th International Conference on Natural Language and Speech Processing (ICNLSP 2024), Trento, Italy, 19–20 October 2024; pp. 439–448.
14. Reid, M.; Savinov, N.; Teplyashin, D.; Lepikhin, D.; Lillicrap, T.; Alayrac, J.b.; Soricut, R.; Lazaridou, A.; Firat, O.; Schrittwieser, J.; et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv* **2024**, arXiv:2403.05530. [CrossRef]
15. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, H. Retrieval-augmented generation for large language models: A survey. *arXiv* **2023**, arXiv:2312.10997.
16. Jin, B.; Liu, G.; Han, C.; Jiang, M.; Ji, H.; Han, J. Large language models on graphs: A comprehensive survey. *arXiv* **2023**, arXiv:2312.02783. [CrossRef]
17. Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; et al. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explor. Newsl.* **2024**, *25*, 42–61. [CrossRef]
18. Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Larson, J. From local to global: A graph rag approach to query-focused summarization. *arXiv* **2024**, arXiv:2404.16130. [CrossRef]
19. Jin, B.; Xie, C.; Zhang, J.; Roy, K.K.; Zhang, Y.; Li, Z.; Li, R.; Tang, X.; Wang, S.; Meng, Y.; et al. Graph chain-of-thought: Augmenting large language models by reasoning on graphs. *arXiv* **2024**, arXiv:2404.07103.
20. Sun, J.; Xu, C.; Tang, L.; Wang, S.; Lin, C.; Gong, Y.; Ni, L.M.; Shum, H.Y.; Guo, J. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. *arXiv* **2023**, arXiv:2307.07697.
21. Shafie, T. A multigraph approach to social network analysis. *J. Soc. Struct. Carnegie Mellon* **2015**, *16*, 1–21. [CrossRef]
22. Francis, N.; Green, A.; Guagliardo, P.; Libkin, L.; Lindaaker, T.; Marsault, V.; Plantikow, S.; Rydberg, M.; Selmer, P.; Taylor, A. Cypher: An evolving query language for property graphs. In Proceedings of the 2018 International Conference on Management of Data, Houston, TX, USA, 10–15 June 2018; pp. 1433–1445.
23. Zhang, Y.; Dai, H.; Kozareva, Z.; Smola, A.J.; Song, L. Variational Reasoning for Question Answering with Knowledge Graph. In Proceedings of the AAAI Conference on Artificial Intelligence, New Orleans, LA, USA, 2–7 February 2018.
24. Papineni, K.; Roukos, S.; Ward, T.; Zhu, W.J. Bleu: A method for automatic evaluation of machine translation. In Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, Philadelphia, PA, USA, 7–12 July 2002; pp. 311–318.
25. Lin, C.Y. Rouge: A package for automatic evaluation of summaries. In Proceedings of the Text Summarization Branches Out, Barcelona, Spain, 25–26 July 2004; pp. 74–81.
26. Wen, T.H.; Vandyke, D.; Mrkšić, N.; Gašić, M.; Rojas-Barahona, L.M.; Su, P.H.; Ultes, S.; Young, S. A Network-based End-to-End Trainable Task-oriented Dialogue System. In Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics, Valencia, Spain, 3–7 April 2017; Lapata, M., Blunsom, P., Koller, A., Eds.; Volume 1, Long Papers, pp. 438–449.
27. Dror, R.; Baumer, G.; Shlomov, S.; Reichart, R. The hitchhiker’s guide to testing statistical significance in natural language processing. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, Melbourne, Australia, 15–20 July 2018; Volume 1: Long papers, pp. 1383–1392.
28. Kwak, S.G.; Kim, J.H. Central limit theorem: The cornerstone of modern statistics. *Korean J. Anesthesiol.* **2017**, *70*, 144–156. [CrossRef]
29. AI@Meta. Llama 3 Model Card. 2024. Available online: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md (accessed on 1 May 2024).
30. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. Gpt-4 technical report. *arXiv* **2023**, arXiv:2303.08774. [CrossRef]
31. Tang, R.; Zhang, X.; Ma, X.; Lin, J.; Ture, F. Found in the middle: Permutation self-consistency improves listwise ranking in large language models. *arXiv* **2023**, arXiv:2310.07712.
32. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837.
33. Team, Q. Qwen2.5: A Party of Foundation Models. 2024. Available online: <https://qwenlm.github.io/blog/qwen2.5/> (accessed on 1 April 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.